

# **STUDENT TASK MANAGER**

*A Full-Stack Web Application for Task Management, Real-Time Collaboration & Analytics*

An **Internship Project Report** submitted in partial fulfillment of the requirements  
for the award of the degree of  
**Bachelor of Technology in Electronics and Communication Engineering**

**PREPARED BY:**

**Bhanothu Manvika**  
**Challa Mallikarjun Reddy**  
**Rayanuddin Mohd**

**FACULTY GUIDE:**

**Dr. K. Narsimha Reddy**  
Project Faculty Guide  
Department of ECE

**JULY 2026**

## **CERTIFICATE**

This is to certify that the internship project report entitled "**STUDENT TASK MANAGER**" is a bonafide work carried out by **Bhanothu Manvika**, **Challa Mallikarjun Reddy**, and **Rayanuddin Mohd** in partial fulfillment of the requirements for the award of **Bachelor of Technology in Electronics and Communication Engineering** during the academic year 2025-2026.

The results embodied in this report have not been submitted to any other University or Institution for the award of any degree or diploma.

**Dr. K. Narsimha Reddy**  
Project Faculty Guide  
Department of ECE

**Dr. S. Rajendar**  
Professor & HOD  
Department of ECE

## ACKNOWLEDGMENTS

---

We express our deep sense of gratitude to our project faculty guide, **Dr. K. Narsimha Reddy**, Department of Electronics and Communication Engineering, for his invaluable guidance, constant encouragement, and constructive feedback throughout the course of this internship project.

We are immensely thankful to **Dr. S. Rajendar**, Professor and Head of the Department of Electronics and Communication Engineering, for providing the necessary facilities and support to complete this work within the campus environment.

Finally, we express our heartfelt appreciation to our parents, friends, and the institutional staff of **Vardhaman College of Engineering** who directly or indirectly assisted us in making this internship project a success.

## ABSTRACT

---

This report presents the design and implementation of **Student Task Manager**, an intelligent full-stack web application developed during the internship program to streamline academic task management, deadline tracking, and collaborative workflow execution for students. Built on a modern web architecture, the platform integrates an intuitive user interface with a high-throughput backend server and real-time database architecture. Equipped with automated notification mechanisms, progress visualizers, and role-based authorization protocols, Student Task Manager serves as both a personal productivity suite and a team collaboration terminal.

The software architecture exploits asynchronous event loops and RESTful API structures to concurrently process live task synchronization, secure JWT authentication, and automated schedule optimization. Experimental results validate low-latency client-server communication, efficient database querying, and robust system stability under multi-user operational conditions during full-stack deployment.

# 1. INTRODUCTION

---

## 1.1 Project Overview

Modern academic environments require students to juggle complex assignments, team projects, and strict submission schedules. Existing task tracking tools are either overly simplified static to-do lists or overly complex enterprise project management suites. **Student Task Manager** bridges this gap by offering a custom-tailored full-stack solution that combines low-friction task tracking with real-time analytics and team collaboration.

## 1.2 Motivation

Static productivity applications lack real-time synchronization and contextual academic workflows. Modern web development frameworks allow for the creation of responsive, highly dynamic applications capable of handling asynchronous data streams, real-time updates, and centralized state management seamlessly across distributed client devices.

## 1.3 Project Objectives

1. **To engineer** a scalable backend API service capable of executing CRUD operations for multi-user task management with minimal latency.
2. **To implement** an interactive dashboard rendering real-time progress analytics and deadline visualizations.
3. **To establish** secure authentication and state management utilizing JSON Web Tokens (JWT) and role-based access control (RBAC).
4. **To integrate** a real-time event pipeline for instantaneous task status updates across collaborative workspaces.

## 2. SYSTEM ARCHITECTURE & MODULE DESIGN

The full-stack application architecture follows a decoupled Client-Server model supported by a structured database layer.

### 2.1 Technology Stack Specifications

| Layer           | Technology / Tool                  | Operational Role in Student Task Manager                                                 |
|-----------------|------------------------------------|------------------------------------------------------------------------------------------|
| Frontend UI     | React.js / HTML5 / CSS3 / Tailwind | Single Page Application (SPA) rendering dynamic user interfaces and state components.    |
| Backend Runtime | Node.js / Express.js Framework     | Microservice application logic handling route endpoints, validation, and business logic. |
| Database Layer  | MongoDB / PostgreSQL               | Persistence storage for user profiles, task metadata, submission statuses, and logs.     |
| Authentication  | JWT & Bcrypt Engine                | Manages stateless session tokens, password hashing, and endpoint security filters.       |

### 2.2 System Interface & API Endpoint Mapping

| Subsystem Endpoint | Protocol / Method | Route Path            | Operational Function                                       |
|--------------------|-------------------|-----------------------|------------------------------------------------------------|
| Auth Subsystem     | POST              | /api/v1/auth/register | Registers new user credentials with encrypted hashing.     |
| Auth Subsystem     | POST              | /api/v1/auth/login    | Authenticates user and issues signed JWT bearer tokens.    |
| Task Management    | GET               | /api/v1/tasks         | Fetches filtered task lists for the active user session.   |
| Task Management    | POST              | /api/v1/tasks/create  | Adds a new task entity with metadata (deadline, priority). |
| Task Management    | PUT               | /api/v1/tasks/:id     | Updates task status, progress metrics, or assignees.       |
| Task Management    | DELETE            | /api/v1/tasks/:id     | Removes task entries with cascade cleanup.                 |

**Critical Architecture Rule:** Backend security filters must validate the Authorization HTTP Header (Bearer <JWT>) on every non-public route. Requests missing valid signature tokens must be terminated immediately with HTTP 401 Unauthorized status to prevent unauthorized access to user data.

## 3. SOFTWARE DESIGN & IMPLEMENTATION

---

### 3.1 Task Prioritization & Analytics Logic

Task sorting and dynamic priority scoring are calculated using a weighted urgency function. The score  $S(t)$  for a task  $t$  is calculated based on its remaining time  $T_{rem}$  (in hours) and raw priority rating  $P$ :

$$S(t) = w_1 \cdot (1 / (T_{rem} + 1)) + w_2 \cdot P$$

Where:

- $w_1, w_2$  are weight coefficients adjusting the balance between deadline proximity and assigned priority.
- $T_{rem}$  represents hours remaining until deadline expiration.
- $P$  represents assigned priority scale ( $1 = Low, 2 = Medium, 3 = High$ ).

### 3.2 Dynamic Route & Authentication Handler

```
// Express.js Route Authentication Middleware Sample
const jwt = require('jsonwebtoken');

const authenticateToken = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) return res.sendStatus(401);

  jwt.verify(token, process.env.ACCESS_TOKEN_SECRET, (err, user) => {
    if (err) return res.sendStatus(403);
    req.user = user;
    next();
  });
};
```

## 4. OPERATIONAL MODES & UI INTERFACE PROFILES

### 4.1 Interface Modes

- **Dashboard Terminal Mode:** Renders analytics charts, pending task counts, completion percentages, and upcoming deadline lists.
- **Workspace Board Mode:** Kanban-style interface displaying tasks across structural stages (To-Do, In Progress, Completed).

### 4.2 State Transition & Action Matrix

| User Command  | Target System State | Application Action                                      | UI Rendering State      |
|---------------|---------------------|---------------------------------------------------------|-------------------------|
| CREATE_TASK   | STATE_MUTATION      | Validates input body and appends entity to database     | Renders new card entry  |
| UPDATE_STATUS | STATE_MUTATION      | Updates task status field and re-indexes priority queue | Dynamic board animation |
| FILTER_VIEW   | STATE_QUERY         | Filters in-memory state by subject or priority tier     | Refreshed task view     |
| EXPORT_REPORT | STATE_READ          | Compiles completion statistics into export format       | File download trigger   |

## 5. RESULTS, CONCLUSIONS & FUTURE ENHANCEMENTS

---

### 5.1 System Verification & Performance Results

Testing confirmed seamless integration between frontend client interfaces and backend microservices. The backend Express routing layer maintained low response times under concurrent API requests. Database indexes successfully optimized query performance during large retrieval operations.

### 5.2 Technical Conclusions

The Student Task Manager project demonstrates the capability of modern full-stack web architectures to deliver responsive, scalable productivity solutions. Decoupling the client interface from core backend API logic ensured high code modularity and ease of maintenance.

### 5.3 Future Scope

- **AI Task Estimator:** Integrating Machine Learning models to predict time required for task completion based on user history.
- **Real-time Collaboration:** Expanding WebSockets support for live document sharing and interactive discussion threads.
- **Mobile Application:** Building a native mobile frontend using React Native to support cross-platform mobile functionality.